JAVA BINARY SEARCH TREE

ALL IMPORTANT INTERVIEW TOPICS • COMPLETE HANDBOOK

From absolute fundamentals to advanced interview patterns

BST operations • Traversals • Queries • Construction • Recovery • Balancing • Range problems • Iterators • Serialization

A practical Java guide with complete implementations, reasoning, dry runs, complexity, edge cases, and a structured interview problem bank.

Learn the invariant. Trace the pointers. Code without looking. Test the edge cases.

How this expanded handbook is organized

Bhai, this edition is deliberately broader than the original topic list. It includes your requested topics plus the other high-value BST concepts commonly used in coding interviews. A topic is not excluded merely because it was not in your first list.

• **Core:** BST rules, representations, traversals, search, insert, delete, min/max.

• **Queries:** validation, kth order statistics, successor/predecessor, floor/ceiling, LCA, range sum.

• **Construction and repair:** sorted array/list to BST, traversal-based construction, recover swapped BST, balance an existing BST.

• **Advanced patterns:** two-sum, greater-sum tree, trim BST, split/join ideas, BST iterator, serialization, Morris traversal.

• **Engineering:** duplicate-key policy, parent pointers, subtree sizes, balanced BSTs, testing and complexity.

BST invariant used here: unique keys; every key in the left subtree is strictly smaller than the node, and every key in the right subtree is strictly greater. If an interview defines duplicate handling differently, state and implement that policy consistently.

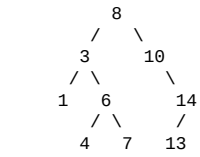
Study order

Phase	Topics
1	Fundamentals, node structure, inorder, search, insert, min/max
2	Delete, validate, kth smallest/largest, successor/predecessor, LCA
3	Sorted array/list construction, recover BST, balance BST
4	Floor/ceiling, range sum, trim, two-sum, greater-sum tree
5	Iterator, serialization, traversal construction, Morris, balancing concepts

1. BST fundamentals and terminology

Definition and ordering rule

A Binary Search Tree is a binary tree organized by key order. For every node x: all keys in x.left are < x.val, and all keys in x.right are > x.val. The condition applies to complete subtrees, not only immediate children.




```
Inorder: 1, 3, 4, 6, 7, 8, 10, 13, 14
```

Terms

• Root: top node; parent: direct predecessor; child: direct descendant.

• Leaf: node with no children; internal node: node with at least one child.

• Subtree: a node and all descendants; ancestor/descendant: upward/downward relationship.

• Height of a node: longest downward path to a leaf; depth: edges from root.

• Balanced: height remains proportional to log n; skewed: tree resembles a linked list.

BST vs binary tree

Every BST is a binary tree, but not every binary tree is a BST. A binary tree only limits each node to at most two children; a BST additionally imposes an ordering invariant.

Height and complexity

Shape	Height	Search/insert/delete
Balanced	O(log n)	O(log n)
Skewed	O(n)	O(n)
Empty	-1 using edge convention	No node visited
Single node	0 using edge convention	O(1)

For a plain BST, always state O(h). Do not promise O(log n) unless balancing is guaranteed.

Why inorder is sorted

Inorder is Left → Node → Right. The left subtree is smaller, then the current key, then the right subtree is larger. This produces strictly increasing values for a unique-key BST.

2. Java node model and base utilities

```
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

Recursive inorder, preorder, postorder

```
static void inorder(TreeNode n) {
    if (n == null) return;
    inorder(n.left);
    System.out.print(n.val + " ");
    inorder(n.right);
}

static void preorder(TreeNode n) {
    if (n == null) return;
    System.out.print(n.val + " ");
    preorder(n.left);
    preorder(n.right);
}

static void postorder(TreeNode n) {
    if (n == null) return;
    postorder(n.left);
    postorder(n.right);
    System.out.print(n.val + " ");
}
```

All three visit each node once: O(n) time and O(h) recursion stack. Inorder is sorted only because this is a BST; preorder and postorder are not sorted.

Iterative inorder

```
static List<Integer> inorderIterative(TreeNode root) {
    List<Integer> out = new ArrayList<>();
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;

    while (cur != null || !st.isEmpty()) {
        while (cur != null) {
            st.push(cur);
            cur = cur.left;
        }
        cur = st.pop();
        out.add(cur.val);
        cur = cur.right;
    }
    return out;
}
```

Level order (BFS)

```
static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> ans = new ArrayList<>();
    if (root == null) return ans;

    Queue<TreeNode> q = new ArrayDeque<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int count = q.size();
        List<Integer> level = new ArrayList<>();

        for (int i = 0; i < count; i++) {
            TreeNode n = q.poll();
            level.add(n.val);
            if (n.left != null) q.offer(n.left);
            if (n.right != null) q.offer(n.right);
        }
        ans.add(level);
    }
    return ans;
}
```

Traversal cost: O(n) time; O(h) stack for DFS, O(w) queue for BFS where w is maximum width.

3. Search in BST

Reasoning

Compare key with current node. Equal means found. Smaller means only left can contain it; larger means only right can contain it.

```
static TreeNode search(TreeNode root, int key) {
    TreeNode cur = root;
    while (cur != null) {
        if (cur.val == key) return cur;
        cur = key < cur.val ? cur.left : cur.right;
    }
    return null;
}

static TreeNode searchRecursive(TreeNode root, int key) {
    if (root == null || root.val == key) return root;
    if (key < root.val) return searchRecursive(root.left, key);
    return searchRecursive(root.right, key);
}
```

Dry run search(7): 8 → left 3 → right 6 → right 7 → found. Time O(h); iterative space O(1), recursive O(h).

Search variations

• Search and return boolean instead of node.

• Find closest value to target: track the node with minimum absolute difference while descending.

• Search in a BST with parent pointers: same comparison path; parent links help upward navigation.

• Search for first key ≥ x (lower bound) or first key > x (upper bound): maintain a candidate.

```
static TreeNode lowerBound(TreeNode root, int x) {
    TreeNode ans = null, cur = root;
    while (cur != null) {
        if (cur.val >= x) { ans = cur; cur = cur.left; }
        else cur = cur.right;
    }
    return ans;
}

static TreeNode upperBound(TreeNode root, int x) {
    TreeNode ans = null, cur = root;
    while (cur != null) {
        if (cur.val > x) { ans = cur; cur = cur.left; }
        else cur = cur.right;
    }
    return ans;
}
```

4. Insert into BST

Descend until the correct null child position. This edition ignores duplicate keys; other policies are possible but must be explicit.

```
static TreeNode insert(TreeNode root, int x) {
    if (root == null) return new TreeNode(x);
    if (x < root.val) root.left = insert(root.left, x);
    else if (x > root.val) root.right = insert(root.right, x);
    return root;
}

static TreeNode insertIterative(TreeNode root, int x) {
    if (root == null) return new TreeNode(x);
    TreeNode cur = root, parent = null;

    while (cur != null) {
        parent = cur;
        if (x < cur.val) cur = cur.left;
        else if (x > cur.val) cur = cur.right;
        else return root; // duplicate ignored
    }

    if (x < parent.val) parent.left = new TreeNode(x);
    else parent.right = new TreeNode(x);
    return root;
}
```

Always write `root = insert(root, x)` because inserting into an empty tree creates a new root. Time O(h); recursive space O(h), iterative O(1).

5. Minimum, maximum, floor and ceiling

```
static TreeNode minNode(TreeNode root) {
    if (root == null) return null;
    while (root.left != null) root = root.left;
    return root;
}

static TreeNode maxNode(TreeNode root) {
    if (root == null) return null;
    while (root.right != null) root = root.right;
    return root;
}

// Floor: greatest value <= x.
static Integer floor(TreeNode root, int x) {
    Integer ans = null;
    while (root != null) {
        if (root.val == x) return x;
        if (root.val < x) { ans = root.val; root = root.right; }
        else root = root.left;
    }
    return ans;
}

// Ceiling: smallest value >= x.
static Integer ceiling(TreeNode root, int x) {
    Integer ans = null;
    while (root != null) {
        if (root.val == x) return x;
        if (root.val > x) { ans = root.val; root = root.left; }
        else root = root.right;
    }
    return ans;
}
```

Min is leftmost; max is rightmost. Floor and ceiling keep the best candidate while searching. All are O(h) time, O(1) extra space.

6. Delete from BST — all three cases

First search for the target, then preserve the BST invariant after removal.

Case A: leaf

Return null; the parent's child reference becomes null.

Case B: exactly one child

Return the non-null child; parent bypasses the removed node. If deleting root, the child becomes the new root.

Case C: two children

Find inorder successor = minimum node in right subtree. Copy its value into target, then delete successor from the right subtree. Successor has no left child, so its removal is now case A or B.

```
static TreeNode delete(TreeNode root, int key) {
    if (root == null) return null;

    if (key < root.val) {
        root.left = delete(root.left, key);
    } else if (key > root.val) {
        root.right = delete(root.right, key);
    } else {
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        TreeNode s = minNode(root.right);
        root.val = s.val;
        root.right = delete(root.right, s.val);
    }
    return root;
}
```

Call as `root = delete(root, key)`. Time O(h), recursion O(h).

Alternative: use inorder predecessor

You may instead copy the maximum value from the left subtree, then delete that predecessor. Both approaches are correct when implemented consistently.

Common deletion bugs

• Forgetting to update root when root itself is removed.

• Removing successor without reconnecting its possible right child.

• Copying successor value but not deleting its old node.

• Using the right child directly as successor even when a smaller node exists deeper in that subtree.

7. Validate BST

Checking only direct children is wrong. A descendant can violate an ancestor's bound.

```
static boolean isValidBST(TreeNode root) {
    return valid(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

static boolean valid(TreeNode n, long low, long high) {
    if (n == null) return true;
    if (n.val <= low || n.val >= high) return false;
    return valid(n.left, low, n.val)
        && valid(n.right, n.val, high);
}
```

Alternative: iterative inorder must be strictly increasing. Use long for previous value so Integer.MIN_VALUE is handled safely.

```
static boolean validInorder(TreeNode root) {
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;
    long prev = Long.MIN_VALUE;

    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.left; }
        cur = st.pop();
        if (cur.val <= prev) return false;
        prev = cur.val;
        cur = cur.right;
    }
    return true;
}
```

Both O(n) time and O(h) auxiliary space. Strict inequalities reject duplicates.

8. Kth smallest and kth largest

```
static Integer kthSmallest(TreeNode root, int k) {
    if (k <= 0) return null;
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;

    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.left; }
        cur = st.pop();
        if (--k == 0) return cur.val;
        cur = cur.right;
    }
    return null;
}

static Integer kthLargest(TreeNode root, int k) {
    if (k <= 0) return null;
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root;

    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.right; }
        cur = st.pop();
        if (--k == 0) return cur.val;
        cur = cur.left;
    }
    return null;
}
```

Inorder ascending gives kth smallest; reverse inorder descending gives kth largest. O(h+k) traversal work in common analysis, worst O(n); O(h) stack.

Subtree-size augmentation

Store `size = 1 + size(left) + size(right)` at every node. At node x, let L=size(x.left): if k=L+1 return x; if k≤L go left; otherwise go right with k-L-1. Query O(h), but insert/delete must maintain sizes correctly.

9. Inorder successor and predecessor

Successor is the smallest key strictly greater than x. Predecessor is the largest key strictly smaller than x. If the key exists: successor is min(right subtree), else it is the lowest ancestor where the node lies in the left branch. Candidate tracking works without parent pointers.

```
static TreeNode successor(TreeNode root, int key) {
    TreeNode ans = null, cur = root;
    while (cur != null) {
        if (key < cur.val) { ans = cur; cur = cur.left; }
        else if (key > cur.val) cur = cur.right;
        else return cur.right != null ? minNode(cur.right) : ans;
    }
    return ans; // next greater even if key is absent
}

static TreeNode predecessor(TreeNode root, int key) {
    TreeNode ans = null, cur = root;
    while (cur != null) {
        if (key > cur.val) { ans = cur; cur = cur.right; }
        else if (key < cur.val) cur = cur.left;
        else return cur.left != null ? maxNode(cur.left) : ans;
    }
    return ans; // next smaller even if key is absent
}
```

O(h) time and O(1) extra space. Successor is not necessarily the right child; it may be an ancestor.

10. Lowest Common Ancestor in BST

```
static TreeNode lcaBST(TreeNode root, int p, int q) {
    TreeNode cur = root;
    while (cur != null) {
        if (p < cur.val && q < cur.val) cur = cur.left;
        else if (p > cur.val && q > cur.val) cur = cur.right;
        else return cur; // split point
    }
    return null;
}
```

If both keys are smaller, move left; if both larger, move right; otherwise current node is the split point. O(h), O(1).

If the problem does not guarantee both keys exist and requires null when either is missing, verify both with search before computing LCA.

BST LCA vs general binary-tree LCA

The split-point shortcut depends on BST ordering. For an arbitrary binary tree, use a postorder recursion or parent-pointer approach; do not apply this BST method blindly.

11. Sorted array to balanced BST

```
static TreeNode sortedArrayToBST(int[] a) {
    return build(a, 0, a.length - 1);
}

static TreeNode build(int[] a, int lo, int hi) {
    if (lo > hi) return null;
    int mid = lo + (hi - lo) / 2;
    TreeNode n = new TreeNode(a[mid]);
    n.left = build(a, lo, mid - 1);
    n.right = build(a, mid + 1, hi);
    return n;
}
```

Choose the middle element as root, recursively build left and right halves. O(n) time to create n nodes; recursion O(log n) for balanced output. Sorted input is required.

Sorted linked list to balanced BST

Repeatedly finding the middle of a linked list costs O(n log n) if each recursive call scans for its middle. An inorder-simulation approach builds in O(n): count list length, recursively build left size, use current list node as root, advance pointer, then build right size.

```
static class ListNode {
    int val; ListNode next;
    ListNode(int v) { val = v; }
}
static ListNode cursor;

static TreeNode sortedListToBST(ListNode head) {
    int n = 0;
    for (ListNode t = head; t != null; t = t.next) n++;
    cursor = head;
    return buildList(0, n - 1);
}

static TreeNode buildList(int lo, int hi) {
    if (lo > hi) return null;
    int mid = lo + (hi - lo) / 2;

    TreeNode left = buildList(lo, mid - 1);
    TreeNode root = new TreeNode(cursor.val);
    cursor = cursor.next;
    root.left = left;
    root.right = buildList(mid + 1, hi);
    return root;
}
```

O(n) time and O(log n) recursion for a balanced result. The static cursor is convenient for a demonstration; in production code, encapsulate it in a builder object to avoid shared mutable state.

12. Recover BST: two values swapped

A valid BST has exactly two node values swapped. Inorder should be increasing, so inversions reveal the misplaced nodes.

```
static void recoverBST(TreeNode root) {
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode cur = root, prev = null;
    TreeNode first = null, middle = null, last = null;

    while (cur != null || !st.isEmpty()) {
        while (cur != null) { st.push(cur); cur = cur.left; }
        cur = st.pop();

        if (prev != null && prev.val > cur.val) {
            if (first == null) {
                first = prev;
                middle = cur;
            } else {
                last = cur;
            }
        }
        prev = cur;
        cur = cur.right;
    }

    if (first != null) {
        TreeNode second = (last != null) ? last : middle;
        int tmp = first.val;
        first.val = second.val;
        second.val = tmp;
    }
}
```

Adjacent swapped nodes produce one inversion: swap first and middle. Non-adjacent swapped nodes produce two inversions: swap first and last. O(n) time, O(h) stack.

Morris traversal for O(1) auxiliary space

Morris inorder temporarily threads the inorder predecessor's right pointer to the current node. It avoids recursion and an explicit stack. Every thread must be removed before traversal ends. Do not return early after finding the inversions.

```
static void recoverBSTMorris(TreeNode root) {
    TreeNode cur = root, prev = null;
    TreeNode first = null, middle = null, last = null;

    while (cur != null) {
        if (cur.left == null) {
            if (prev != null && prev.val > cur.val) {
                if (first == null) { first = prev; middle = cur; }
                else last = cur;
            }
            prev = cur;
            cur = cur.right;
        } else {
            TreeNode pred = cur.left;
            while (pred.right != null && pred.right != cur) pred = pred.right;

            if (pred.right == null) {
                pred.right = cur;
                cur = cur.left;
            } else {
                pred.right = null;
                if (prev != null && prev.val > cur.val) {
                    if (first == null) { first = prev; middle = cur; }
                    else last = cur;
                }
                prev = cur;
                cur = cur.right;
            }
        }
    }

    if (first != null) {
        TreeNode second = last != null ? last : middle;
        int tmp = first.val; first.val = second.val; second.val = tmp;
    }
}
```

O(n) time and O(1) auxiliary space, assuming the traversal completes and restores all temporary links.

13. Balance an existing BST

A plain BST may be skewed. To rebalance without changing key order: perform inorder traversal to collect sorted values, then build a balanced BST by choosing middle elements recursively.

```
static void collect(TreeNode n, List<Integer> a) {
    if (n == null) return;
    collect(n.left, a);
    a.add(n.val);
    collect(n.right, a);
}

static TreeNode balanceBST(TreeNode root) {
    List<Integer> a = new ArrayList<>();
    collect(root, a);
    return buildBalanced(a, 0, a.size() - 1);
}

static TreeNode buildBalanced(List<Integer> a, int lo, int hi) {
    if (lo > hi) return null;
    int mid = lo + (hi - lo) / 2;
    TreeNode n = new TreeNode(a.get(mid));
    n.left = buildBalanced(a, lo, mid - 1);
    n.right = buildBalanced(a, mid + 1, hi);
    return n;
}
```

O(n) time, O(n) list memory, O(log n) recursion after the sorted list is built. This is rebuilding, not an online self-balancing insertion algorithm.

Self-balancing BST families

Structure	Main idea	Typical guarantee
AVL Tree	Strict height balance; rotations	O(log n) search/insert/delete
Red-Black Tree	Color invariants + rotations/recoloring	O(log n) search/insert/delete
Treap	BST order + heap priority	Expected O(log n) with random priorities
Splay Tree	Move accessed node to root	Amortized O(log n)

Know the purpose and guarantees. If the interview asks specifically for AVL or Red-Black implementation, it is a separate deeper topic involving rotations and invariant restoration.

14. Range sum, trim BST and range reporting

Range sum

```
static long rangeSum(TreeNode root, int low, int high) {
    if (root == null) return 0;
    if (root.val < low) return rangeSum(root.right, low, high);
    if (root.val > high) return rangeSum(root.left, low, high);
    return root.val
        + rangeSum(root.left, low, high)
        + rangeSum(root.right, low, high);
}
```

Prune the side that cannot contain an in-range key. Worst case O(n), but pruning can skip large subtrees. `long` avoids sum overflow for many int nodes.

Trim BST to interval [low, high]

```
static TreeNode trimBST(TreeNode root, int low, int high) {
    if (root == null) return null;
    if (root.val < low) return trimBST(root.right, low, high);
    if (root.val > high) return trimBST(root.left, low, high);
    root.left = trimBST(root.left, low, high);
    root.right = trimBST(root.right, low, high);
    return root;
}
```

If root is too small, its left subtree is also too small and can be discarded. If root is too large, its right subtree can be discarded.

Report all keys in a range

```
static void printRange(TreeNode root, int low, int high) {
    if (root == null) return;
    if (root.val > low) printRange(root.left, low, high);
    if (root.val >= low && root.val <= high) System.out.print(root.val + " ");
    if (root.val < high) printRange(root.right, low, high);
}
```

Output-sensitive: O(h + k) in a balanced BST for k reported values, with traversal details depending on tree shape; worst-case O(n).

15. Two Sum in BST

Given target T, find two distinct nodes whose values sum to T.

Approach A: inorder list + two pointers

```
static boolean twoSumBST(TreeNode root, int target) {
    List<Integer> a = new ArrayList<>();
    collectInorder(root, a);

    int i = 0, j = a.size() - 1;
    while (i < j) {
        long sum = (long) a.get(i) + a.get(j);
        if (sum == target) return true;
        if (sum < target) i++;
        else j--;
    }
    return false;
}

static void collectInorder(TreeNode n, List<Integer> a) {
    if (n == null) return;
    collectInorder(n.left, a);
    a.add(n.val);
    collectInorder(n.right, a);
}
```

O(n) time and O(n) memory. Cast to long before addition to avoid integer overflow.

Approach B: two BST iterators

Use one stack iterator that yields ascending values (inorder) and another that yields descending values (reverse inorder). Move the smaller iterator forward when sum is too small; move the larger iterator backward when sum is too large. This can use O(h) memory and stop early.

16. BST Iterator

A common design question: implement `next()` and `hasNext()` so values are returned in ascending order without storing the entire traversal.

```
class BSTIterator {
    private Deque<TreeNode> st = new ArrayDeque<>();

    BSTIterator(TreeNode root) {
        pushLeft(root);
    }

    private void pushLeft(TreeNode n) {
        while (n != null) {
            st.push(n);
            n = n.left;
        }
    }

    boolean hasNext() {
        return !st.isEmpty();
    }

    int next() {
        TreeNode n = st.pop();
        pushLeft(n.right);
        return n.val;
    }
}
```

Each node is pushed and popped once: O(n) total over full iteration, amortized O(1) per `next()`, O(h) space. Reverse iterator uses right-spine pushes.

17. Greater Sum Tree / Convert BST

For each node, replace its value with the sum of all original values greater than or equal to it. Traverse in reverse inorder (Right → Node → Left), maintaining a running sum.

```
static int runningSum = 0;

static TreeNode convertToGreaterTree(TreeNode root) {
    runningSum = 0;
    convert(root);
    return root;
}

static void convert(TreeNode n) {
    if (n == null) return;
    convert(n.right);
    runningSum += n.val;
    n.val = runningSum;
    convert(n.left);
}
```

O(n) time, O(h) recursion. This version uses int; use long if accumulated sums may exceed int. For duplicate keys, define whether equal values are grouped or processed individually.

18. Construct BST from traversal orders

BST from preorder

A preorder sequence for a BST can be reconstructed using a lower/upper bound recursion or a stack. The following bound method assumes unique keys.

```
static int index;

static TreeNode bstFromPreorder(int[] pre) {
    index = 0;
    return buildPre(pre, Long.MIN_VALUE, Long.MAX_VALUE);
}

static TreeNode buildPre(int[] pre, long low, long high) {
    if (index == pre.length) return null;
    int x = pre[index];
    if (x <= low || x >= high) return null;

    index++;
    TreeNode n = new TreeNode(x);
    n.left = buildPre(pre, low, x);
    n.right = buildPre(pre, x, high);
    return n;
}
```

O(n) time; O(h) recursion. After building, ensure all input values were consumed if validating that the entire sequence is a valid preorder.

BST from postorder

Process postorder from right to left: root, then right subtree, then left subtree. Use bounds similarly, reversing the child order.

Build general binary tree from inorder + preorder/postorder

This is a general binary-tree construction problem, not uniquely a BST operation. With unique values, a hashmap of inorder positions enables O(n) construction. Preorder+inorder or postorder+inorder determine the tree; preorder+postorder alone generally does not uniquely determine it.

19. Serialize and deserialize BST

BST serialization can be more compact than general binary-tree serialization because the BST ordering lets us reconstruct structure from preorder or postorder with bounds. A simple safe approach is to store preorder values and rebuild with bounds.

```
static class PreBuilder {
    int i = 0;
    int[] a;

    PreBuilder(int[] a) { this.a = a; }

    TreeNode build(long low, long high) {
        if (i == a.length) return null;
        int x = a[i];
        if (x <= low || x >= high) return null;

        i++;
        TreeNode n = new TreeNode(x);
        n.left = build(low, x);
        n.right = build(x, high);
        return n;
    }
}
```

A production serializer should define a format, handle empty trees, reject malformed input, and verify all tokens are consumed. If arbitrary binary trees (not BSTs) are serialized, include null markers or use a level-order format.

BST vs general-tree serialization

Type	Typical representation	Reconstruction
BST	Preorder/postorder values + ordering bounds	Bounds enforce left/right ranges
General binary tree	Preorder with null markers or level order	Null markers preserve missing-child shape

20. Parent pointers and successor

Some implementations store a parent reference in each node. This makes upward navigation direct.

```
class ParentNode {
    int val;
    ParentNode left, right, parent;
    ParentNode(int v) { val = v; }
}

static ParentNode successorWithParent(ParentNode x) {
    if (x == null) return null;

    if (x.right != null) {
        ParentNode n = x.right;
        while (n.left != null) n = n.left;
        return n;
    }

    ParentNode p = x.parent;
    while (p != null && x == p.right) {
        x = p;
        p = p.parent;
    }
    return p;
}
```

If a right subtree exists, take its minimum. Otherwise climb while current node is a right child; first ancestor where current came from the left is successor. Predecessor is symmetric.

21. Duplicate keys: choose a policy

There is no single universal duplicate policy. Interviewers may define duplicates as forbidden, stored as a count, always placed on one side, or represented as a frequency field.

Policy	Rule	Validation implication
No duplicates	Ignore or reject equals	Strict lower < node < upper
Count field	One node stores frequency	Ordering remains strict between distinct keys
Duplicates always left	Equal values allowed in left side	Use inclusive upper bound on left
Duplicates always right	Equal values allowed in right side	Use inclusive lower bound on right

Do not use a strict validator with an insertion method that places duplicates. The insertion and validation policies must agree.

22. Complexity master table

Operation	Time	Auxiliary space
Search / insert / delete	O(h)	O(1) iterative; O(h) recursive
Min / max / floor / ceiling	O(h)	O(1)
Validate	O(n)	O(h)
Kth order statistic	O(h+k), worst O(n)	O(h)
Successor / predecessor	O(h)	O(1)
LCA in BST	O(h)	O(1)
Sorted array to BST	O(n)	O(log n) recursion + output
Sorted linked list to BST	O(n)	O(log n) recursion + output
Recover BST with stack	O(n)	O(h)
Recover BST with Morris	O(n)	O(1)
Balance by rebuilding	O(n)	O(n) list + recursion
Range sum / trim	O(n) worst	O(h)
BST iterator	O(1) amortized next	O(h)
Two Sum with list	O(n)	O(n)
Two Sum with iterators	O(n) worst	O(h)

h = height. A plain BST can have h=O(n); a balanced BST has h=O(log n).

23. Testing checklist and common interview traps

• Empty tree and one-node tree.

• Delete root in all possible child configurations.

• Delete missing key; tree must remain unchanged.

• Delete a two-child node where successor itself has a right child.

• Validate a deep descendant that violates an ancestor's bound.

• Validate Integer.MIN_VALUE and Integer.MAX_VALUE; use long bounds.

• k <= 0 and k > number of nodes for kth queries.

• Successor of maximum and predecessor of minimum return no node.

• LCA when one or both values are absent: clarify contract.

• Duplicate values: state policy and keep all methods consistent.

• Sorted insertion into plain BST can make it skewed.

• Recover BST: adjacent and non-adjacent swapped values are different inversion cases.

• Morris traversal: finish cleanup; do not early-return while temporary threads exist.

• Use long for sums where int overflow is possible.

Interview explanations to practice

Delete: “I search down one path. A leaf becomes null; a one-child node is replaced by its child; a two-child node is replaced by its inorder successor, which I then remove from its original location. Complexity is O(h).”

Validate: “I propagate an allowed open interval. Moving left tightens the upper bound; moving right tightens the lower bound. This catches deep violations and long bounds support integer extremes.”

Kth smallest: “Inorder is sorted. I perform iterative inorder, decrement k at each visited node, and return when k reaches zero.”

Recover: “The swapped values create one or two inversions in inorder. I identify the first misplaced predecessor and either the middle or last misplaced node, then swap their values.”

24. Full interview problem roadmap

Tier	Problem / variation	Skill tested
Foundation	Search, insert, min/max, inorder	BST invariant
Foundation	Delete node	Pointer reconnection
Core	Validate BST	Global constraints
Core	Kth smallest/largest	Ordered traversal
Core	Successor/predecessor	Candidate + subtree extreme
Core	LCA in BST	Split-point property
Core	Sorted array/list to BST	Divide and conquer
Core	Recover swapped BST	Inorder inversions
Intermediate	Floor, ceiling, lower/upper bound	Candidate tracking
Intermediate	Range sum / report range	Pruning
Intermediate	Trim BST	Subtree elimination
Intermediate	Two Sum in BST	Two iterators
Intermediate	BST Iterator	Lazy inorder
Intermediate	Greater Sum Tree	Reverse inorder
Intermediate	BST from preorder/postorder	Bounds + traversal
Intermediate	Serialize / deserialize BST	Reconstruction
Advanced	Balance existing BST	Inorder + rebuild
Advanced	Subtree-size kth queries	Augmented metadata
Advanced	Morris traversal / recovery	O(1) auxiliary space
Advanced	AVL / Red-Black tree concepts	Self-balancing invariants

Final self-test

- Can you write search, insert, and delete without notes?
- Can you draw all three delete cases and explain successor removal?
- Can you reject a deep invalid node using bounds?
- Can you find kth smallest without sorting the tree?
- Can you find successor when the target has no right subtree?
- Can you explain why BST operations are O(h), not always O(log n)?
- Can you recover adjacent and non-adjacent swapped values?
- Can you explain the difference between rebuilding a balanced BST and maintaining an AVL/Red-Black tree?

Completion target: Be able to implement every core method from memory, trace the advanced patterns, and clearly state assumptions about duplicates, missing keys, and integer overflow.